

DOCUMENTATION OF THE SM LANGUAGE

AVRAHAM AIZENBUD AND MEIR AIZENBUD

The language is based on the vm language from [\[NS05\]](#).

1. SYMBOLS

for the sake of this document a symbol is a string that starts with a letter and might include letters, digits and the character “.”

2. VARIABLES

The SM language uses symbolic variables. A variable name should be symbol. All variables occupy 1 register in the RAM There are of 2 types:

2.1. **local variables.** variables that are used in a function and disappear when the function is finished. Can not be accessed from outside the function. those variables includes the arguments of the function and the internal variables. internal variables must be declared together with the function. All the local variables of a given function should have distinct names.

2.2. **global variables.** variables that are recognized by the interpreter code. All global variables should have distinct names, however they can coincide with names of local variables.

3. STACK

The SM have a global stack. The stack operations (push/pop) always refers to it.

4. FUNCTIONS

All functions including built-in functions are performed on the top of the stack. A function uses how many arguments that it needs from the top of the stack, eliminate those arguments from the stack, and push the result to the stack. The function might have also side effects that influence other operations in the RAM.

4.1. **Functions declaration.** declaration of a function must include all its arguments and its internal variables. Those will be used by the function. The name of a non built-in function must be a symbol. Non built-in functions have to return exactly one result. The result should be on the top of the stack before the return command.

5. LABELS

labels have name that are recognize inside a given function. All the labels in a given function must be distinct, but they can coincide with variables. As variables they represented by a symbol.

6. LOGICAL OPERATION

When performing logical operator, the boolean value of a variable is determined by its most significant bit.

7. INITIALISATION

The first function that will be run should have the name "Sys.init". After its completion, the program will go into infinite loop. Only functions that are called from "Sys.init" (in the hereditary seance) will be performed.

8. FILES

The code might be separated into multiple files. Each file must start with a functions declaration, and it have to consist of (1 or more) functions.

9. COMMANDS

Below is a list of all SM commands.

Note that the strings x , $\sin$, $loop$, $x1$, $x2$, $y1$, $y2$ are given as examples and can be replaced by any symbol.

The string "5" is given as examples and can be replaced by any integer

Three dots "..." means etc.

9.1. Stack operations.

- $- > @x$
pop to the local variable x from the stack
- $- > x$
pop to a global variable x from the stack
- $< -@x$
push the local variable x to the stack
- < -5
push the constant 5 to the stack
- $< -x$
push the global variable x to the stack

9.2. Bult in functions.

9.2.1. *Arithmetic/Logical functions.*

- +
adding the top 2 elements of the stack and replace them by there sum.
- −
substantiating the top element of the stack from the previous one and replace both of them by there sum.
- (−)
replace the top element of the stack with its opposite.
- &
Perform AND on the top 2 elements of the stack and replace them by the result.
(only the moset sugnificant bit are mater)
- |
Perform OR on the top 2 elements of the stack and replace them by the result.
(only the moset sugnificant bit are mater)
- ~
Perform Not on the top 2 elements of the stack and replace them by the result.
(only the moset sugnificant bit are mater)
- ==
Denote the values top elements in the stack by x and y (y is the top one). The func-
tion replcae the top to elements in the stack by a register whos most dugnificant
bit it the troth value of “ $x = y$ ”
- >
Denote the values top elements in the stack by x and y (y is the top one). The func-
tion replcae the top to elements in the stack by a register whos most dugnificant
bit it the troth value of “ $x > y$ ”
- <
Denote the values top elements in the stack by x and y (y is the top one). The func-
tion replcae the top to elements in the stack by a register whos most dugnificant
bit it the troth value of “ $x < y$ ”

9.2.2. *Memory functions.*

- []
Peek function: Denote the value top element in the stack by x. The function
replace the top element in the stack by the content of regester with adress x
- − > []
Pock function. Denote the values top elements in the stack by x and y (y is the top
one). The function eliminate the top 2 elements of the stack and put the valure x
in regester with adress y

9.3. **Flow control.**

- *sin*
call the function sin and apply it on the last elements of the stack. at the and of
the functions they will be replaced by the returned value

- *loop* :
a label with name *loop*. recognized within the function
- *-- > loop*
jump to a label with name *loop*.
- *? -- > loop*
jump to a label with name *loop*, if the top of the stack is true.
- *!sin(x1, x2, ...)y1, y2, ...*
declare a function whose name is *sin* and arguments names (recognized inside the function) are *x1, x2, ...* also the function is allowed to have internal variable with names *y1, y2, ...*
- *< --*
return from a function. All arguments, internal variables and anything that stacked above them (except of the top of the stack) will be eliminated from the stack, and will be replaced by the element that was on the top of the stack before the command.

REFERENCES

- [NS05] N. Nisan and S. Schocken, *The Elements of Computing Systems: Building a Modern Computer from First Principles*, The MIT press 2005.

AVRAHAM AIZENBUD, FACULTY OF MATHEMATICS AND COMPUTER SCIENCE, WEIZMANN INSTITUTE OF SCIENCE, ISRAEL.

Email address: `aizenr@gmail.com`

URL: <http://www.aizenbud.org>

MEIR AIZENBUD,

Email address: `meir.aizenbud@gmail.com`